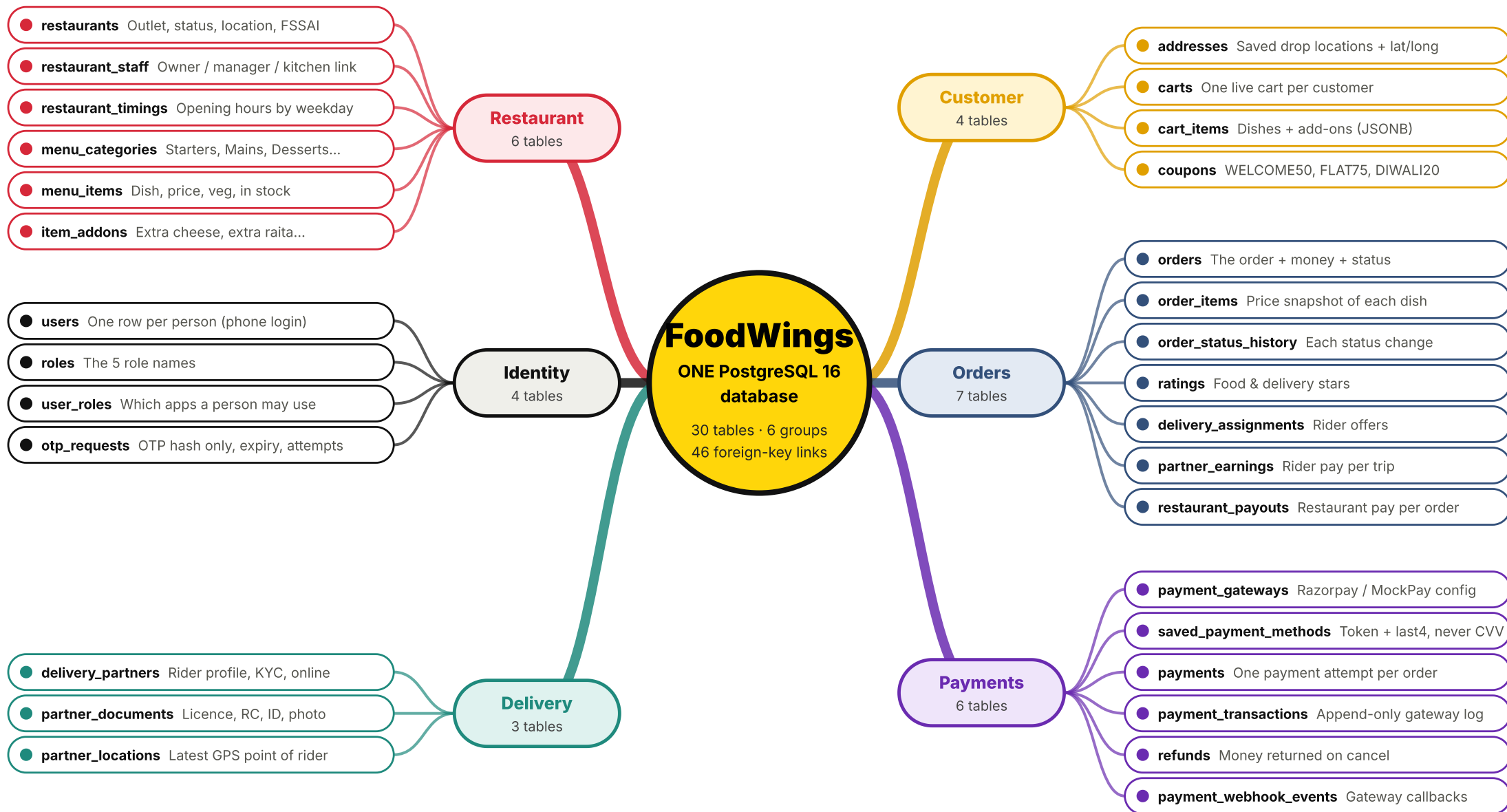




Mind map: the 30 tables at a glance

Every table belongs to one of six groups. The three apps (customer, restaurant, rider) all read and write this one database.

FoodWings · Database guide 1 / 5



3 apps, 1 login table: apps a person can use = rows in user_roles

CUSTOMER

RESTAURANT_OWNER / STAFF

DELIVERY_PARTNER

ADMIN

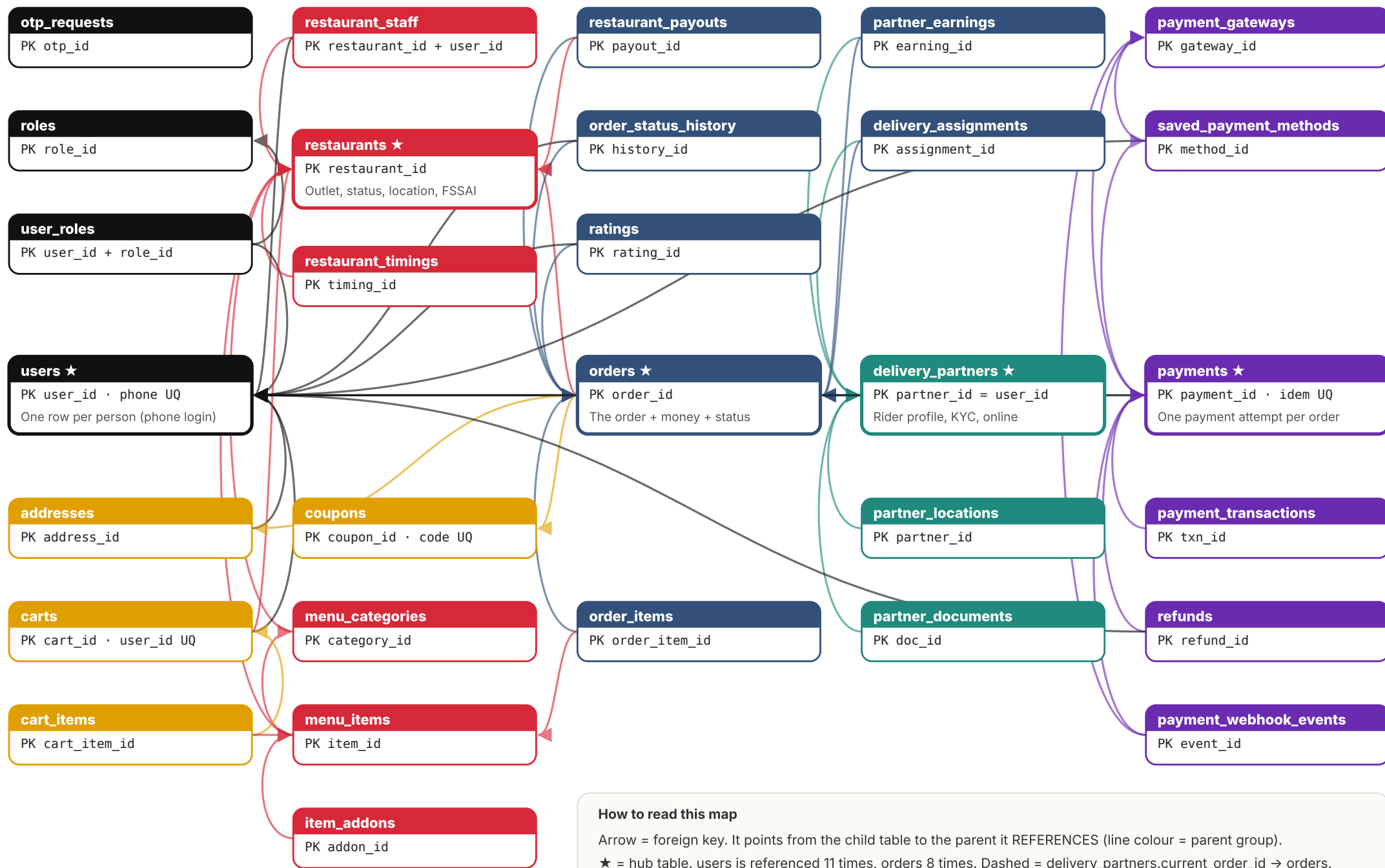
Same user_id can hold several roles (seed user 105 is customer + rider)



How the tables are interlinked

All 30 tables with their primary keys and every foreign key from db/01_schema.sql. A row cannot point to something that does not exist.

FoodWings · Database guide 2 / 5





Why PostgreSQL was chosen

Each need of a food-delivery system on the left is answered by a built-in PostgreSQL feature on the right.

What FoodWings needs

1. Money is never half-saved

Order + items + payment are written together or not at all.

2. Every link must be real

An order must point to a real customer, restaurant and address.

3. Bad values stopped at the door

Status lists, phone format, rating 1–5, price > 0.

4. CVV must never be stored

Not in any table, not in any log file.

5. Two riders tap Accept together

Only one may win the order.

6. Some data is flexible

Chosen add-ons, cuisines, gateway payloads.

PostgreSQL

answers all six
in ONE engine

What PostgreSQL gives

ACID transactions

BEGIN ... COMMIT, row locks (SELECT ... FOR UPDATE)

Foreign keys

46 REFERENCES links, enforced by the database

CHECK constraints

CHECK (status IN (...)), CHECK (rating 1–5)

Event + row triggers

block_card_secret_columns + no values in logs

Partial unique index

UNIQUE (order_id) WHERE status = 'ACCEPTED'

JSONB columns

cart_items.selected_addons JSONB

Need in this project	PostgreSQL ✓ chosen	MongoDB	MySQL
Multi-table ACID transactions	Yes, the core design	Yes (since 4.0), but data is designed to live in one document	Yes (InnoDB)
Foreign keys enforced by the DB	Yes	No: the application must check links	Yes
CHECK constraints	Yes	Schema validation rules (JSON Schema)	Yes (since 8.0.16)
Partial unique index (one ACCEPTED rider)	Yes	Yes (partial index)	No: needs a workaround
Event trigger to block a column named cvv	Yes	No (a validator can forbid a field)	No
Flexible JSON with indexes	JSONB + GIN index	Native (documents)	JSON + generated-column indexes
Location queries later	PostGIS extension	Geo indexes built in	Spatial types (basic)
Cost / licence	Free, open source	Community free; SSPL licence	Free (GPL); Oracle-owned

Why one database, not four?

Orders, payments, users and menus are tightly linked and must stay consistent. Splitting them across MongoDB, Redis and Elasticsearch from day one means four systems to run, back up and keep in sync.

Start: PostgreSQL only (all 30 tables).

Scale later, only if needed: Redis for live rider GPS & carts, Elasticsearch for dish search. PostgreSQL stays the **source of truth**.

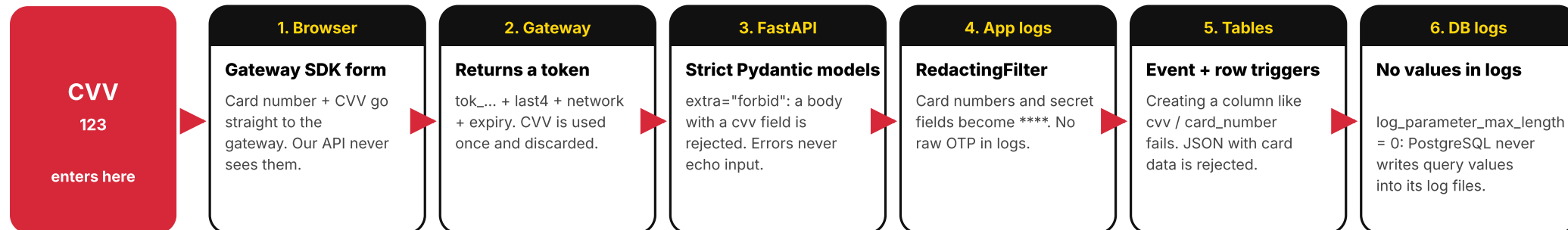


The important features

How the database keeps card data out, money exact, and orders consistent, with the exact feature used for each job.

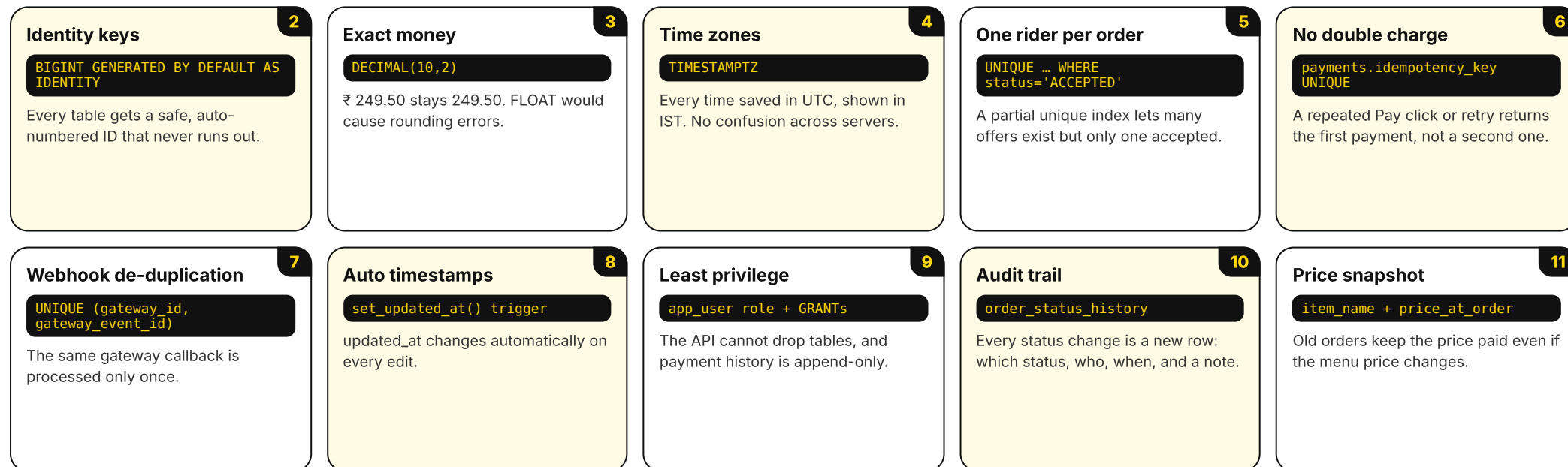
FoodWings · Database guide 4 / 5

Feature 1 · The CVV rule, enforced at 6 layers (defence in depth)



Stored at the end: only token · last4 · network · expiry ✓ Never stored: card number · CVV ✗

Features 2-11 · Built-in PostgreSQL features this design relies on

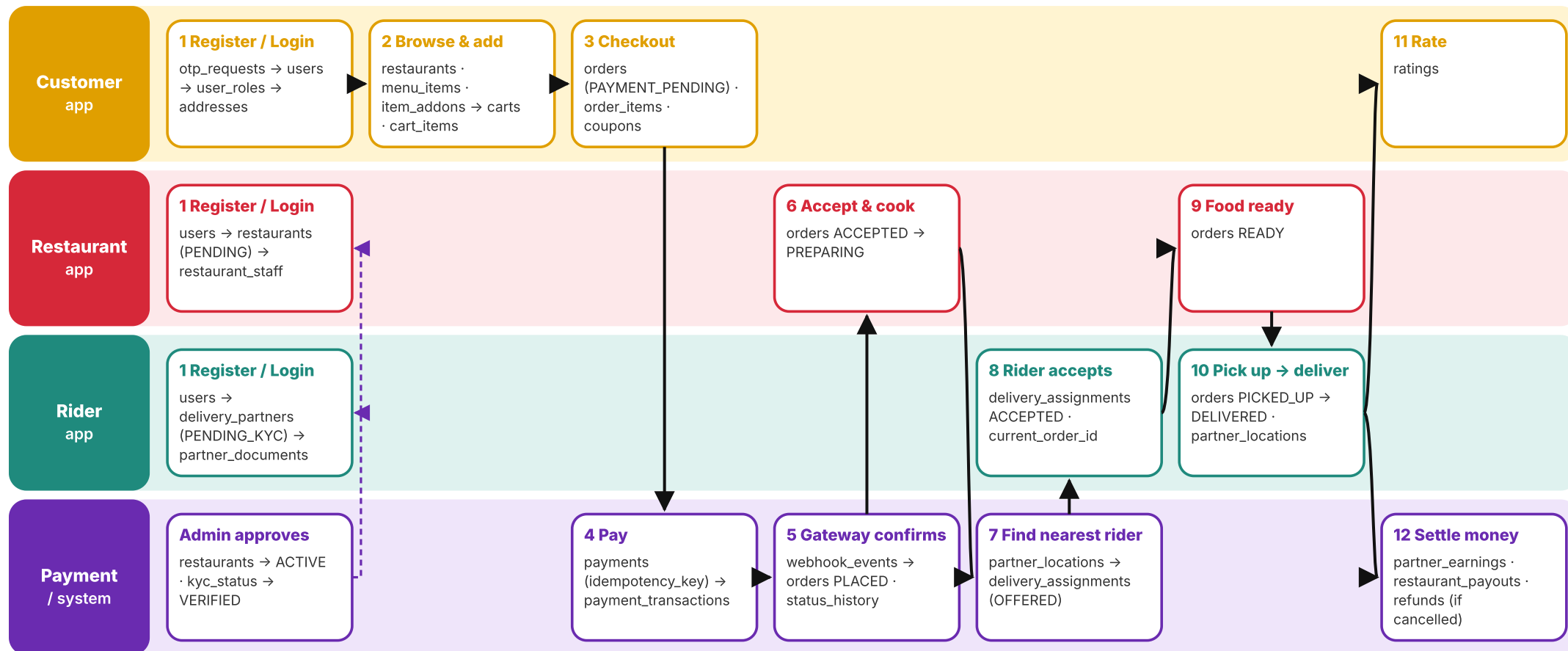


Where to see it in the code: db/01_schema.sql (tables, keys, CHECKs, indexes) · db/02_payment_security.sql (CVV triggers, log settings, app_user grants) · api/app/logging_setup.py (RedactingFilter) · api/app/schemas.py (strict models) · api/app/services/order_flow.py (allowed status moves)



The workflow: from registration to payment to delivery

Each box shows the step and the tables it writes. The same user_id, restaurant_id and order_id travel through all three apps.



orders.status: the only allowed moves (checked in order_flow.py, every move logged in order_status_history)

